Mahidol University
Faculty of Information
and Communication Technology

Software Security and Buffer Overflow

Ittipon (Art) Rassameeroj

Agenda

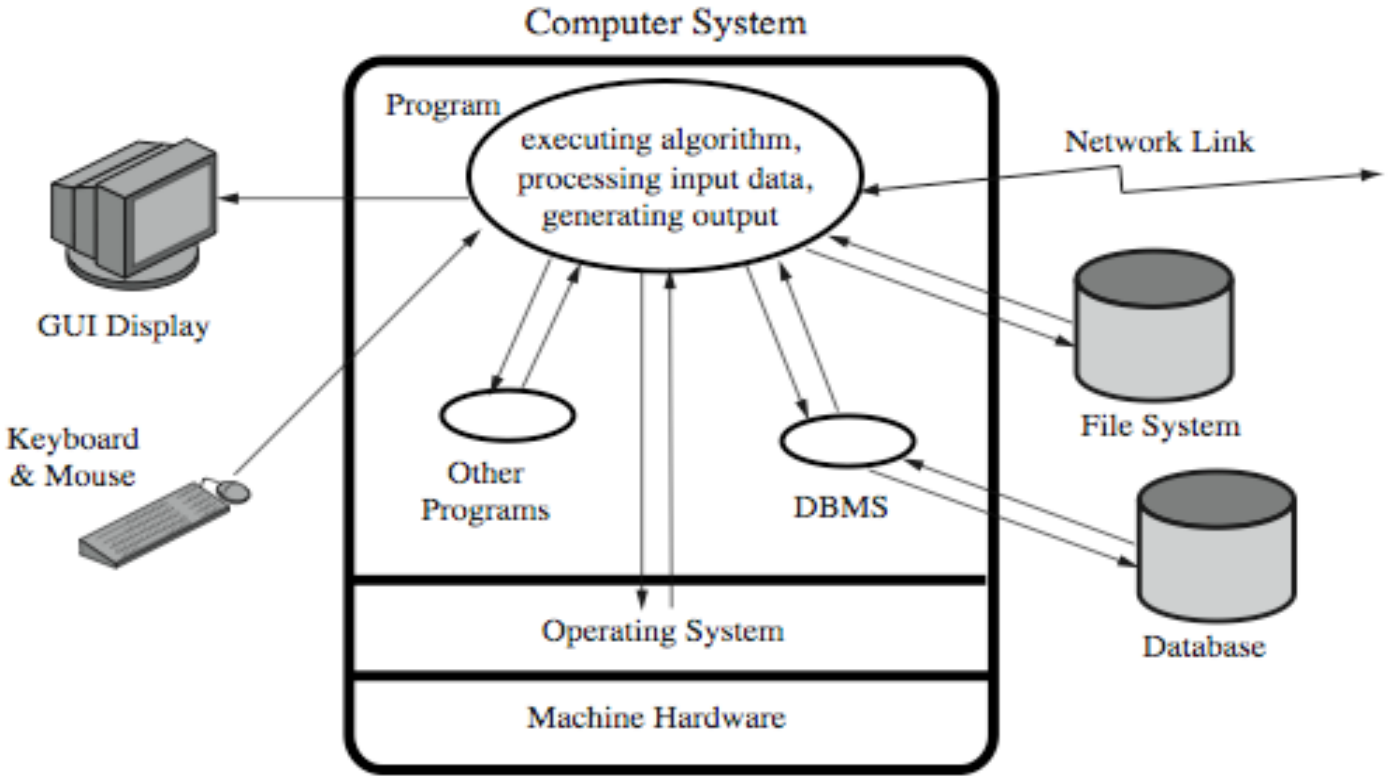
Software security (Chapter 11)

- Causes of software vulnerabilities
- Software security issues
- Software qualify vs security
- Secure coding

Buffer overflow (Chapter 10)

- Basic and attacks
- Stack overflow
- Heap overflow
- Defense

Abstract View of Program



Software Vulnerabilities

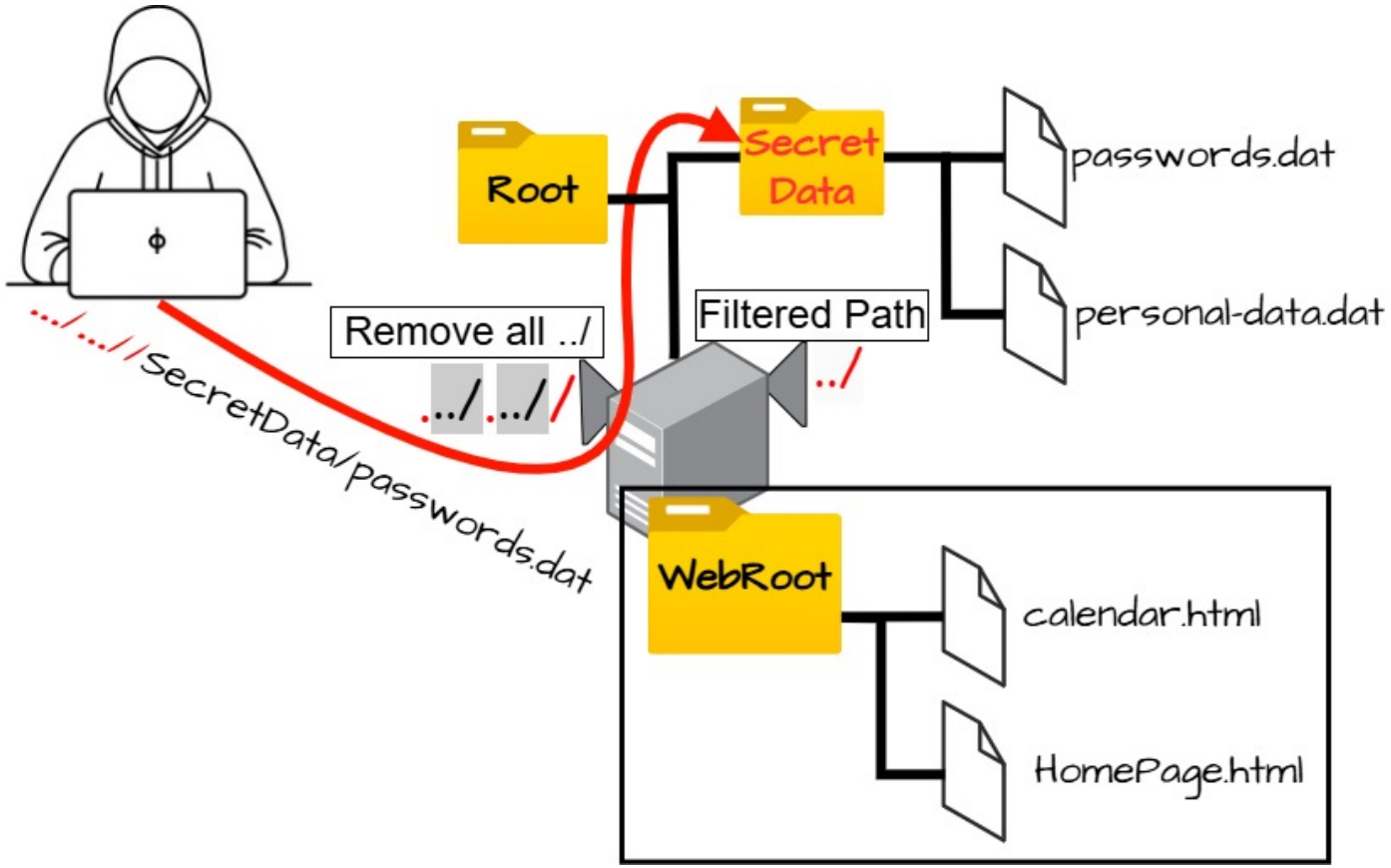
- Security failures can result from intentional or non-malicious causes; both can cause harm.
- Programming oversights: buffer overflows, off-by-one errors, incomplete mediation, time-of-check to time-of-use errors.
- A problem can reside anywhere between the machine hardware and the user interface.
- Attackers can exploit a non-malicious flaw to cause real harm.
- Many vulnerabilities result from “poor programming”.

Top 25 Most Dangerous Software Errors

Rank	ID	Name	Score	CVEs in KEV	Rank Change vs. 2024
1	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')	60.38	7	0
2	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	28.72	4	+1
3	CWE-352	Cross-Site Request Forgery (CSRF)	13.64	0	+1
4	CWE-862	Missing Authorization	13.28	0	+5
5	CWE-787	Out-of-bounds Write	12.68	12	-3
6	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')	8.99	10	-1
7	CWE-416	Use After Free	8.47	14	+1
8	CWE-125	Out-of-bounds Read	7.88	3	-2
9	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')	7.85	20	-2
10	CWE-94	Improper Control of Generation of Code ('Code Injection')	7.57	7	+1
11	CWE-120	Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')	6.96	0	N/A
12	CWE-434	Unrestricted Upload of File with Dangerous Type	6.87	4	-2
13	CWE-476	NULL Pointer Dereference	6.41	0	+8
14	CWE-121	Stack-based Buffer Overflow	5.75	4	N/A
15	CWE-502	Deserialization of Untrusted Data	5.23	11	+1
16	CWE-122	Heap-based Buffer Overflow	5.21	6	N/A
17	CWE-863	Incorrect Authorization	4.14	4	+1
18	CWE-20	Improper Input Validation	4.09	2	-6
19	CWE-284	Improper Access Control	4.07	1	N/A
20	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor	4.01	1	-3
21	CWE-306	Missing Authentication for Critical Function	3.47	11	+4
22	CWE-918	Server-Side Request Forgery (SSRF)	3.36	0	-3
23	CWE-77	Improper Neutralization of Special Elements used in a Command ('Command Injection')	3.15	2	-10
24	CWE-639	Authorization Bypass Through User-Controlled Key	2.62	0	+6
25	CWE-770	Allocation of Resources Without Limits or Throttling	2.54	0	+1

Source: https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.htm

Path Traversal Example



https://cwe.mitre.org/data/definitions/35.html

Software Security vs Quality

- Software quality and reliability are concerned with accidental failure of program, mostly resulting from
  - Random, unanticipated input
  - System interaction
  - Use of incorrect code
- Software security is difference
  - Attacker chooses specific target of buggy code to exploit
  - Often triggered by very unlikely inputs
  - Common tests couldn't identify those bugs
- To improve software security, pay attention to all aspects of
  - How program executes
  - Execution environment
  - Type of data processed

Secure Coding

- Secure coding is the process of designing and implementing software in the way that guards against the accidental introduction of security vulnerabilities.
- Reduce or eliminate the prospect of “Murphy’s law: anything can go wrong”.
- Assume nothing e.g., a particular function call “will” work as advertised.
- Check all potential errors not just focusing on solving task.
- Must validate all assumptions.
- All potential failures must be handled gracefully and safely.

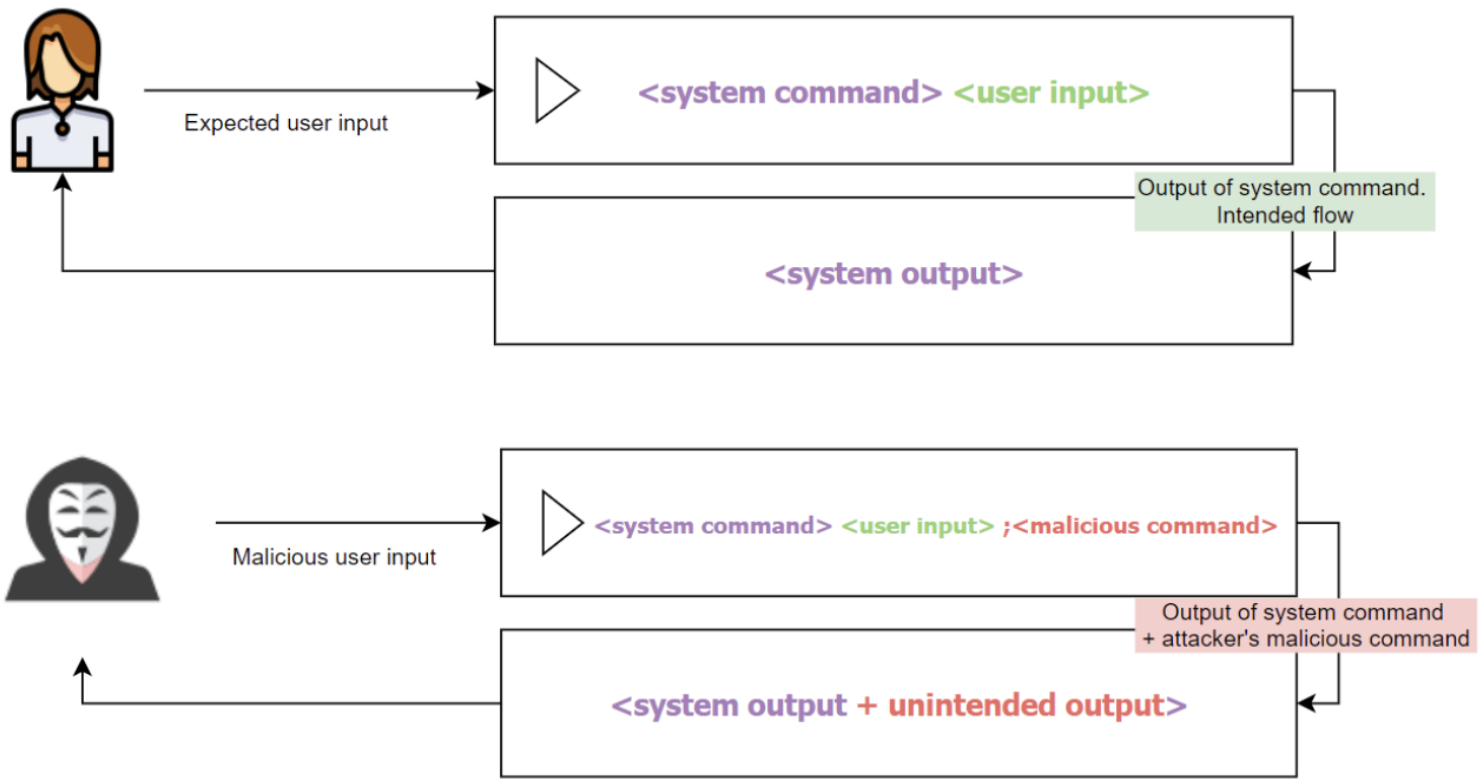
Consideration of Secure Coding

- Handling program input
- Writing safe code
- Interacting with OS and other programs
- Handling program output

Handling Program Input

- Incorrect handling a very common failure. In real environment, input is any sources of data from outside.
- Therefore, we must identify all data sources explicitly validate assumptions on size and type of values before use.
  - Validate size of input
  - Validate type of input
  - Validate Interpretation of input
  - Validating input syntax
  - Input fuzzing

Command Injection



https://www.stackhawk.com/blog/nodejs-command-injection-examples-and-prevention/

Command Injection Example

List Folders

/usr | Submit

```
total 88
drwxr-xr-x   2 root root 36864 Sep  6 08:27 bin
drwxr-xr-x   2 root root  4096 Apr 15  2020 games
drwxr-xr-x  36 root root  4096 Aug  4 15:27 include
drwxr-xr-x  77 root root  4096 Aug  1 23:15 lib
drwxr-xr-x   2 root root  4096 Aug  5  2020 lib32
drwxr-xr-x   2 root root  4096 Mar 20 15:24 lib64
drwxr-xr-x   4 root root  4096 Feb  3  2021 libexec
drwxr-xr-x   2 root root  4096 Aug  5  2020 libx32
drwxr-xr-x  12 root root  4096 Feb 16  2021 local
drwxr-xr-x   2 root root 12288 Aug  1 23:15 sbin
drwxr-xr-x 119 root root  4096 Aug  4 15:27 share
drwxr-xr-x   2 root root  4096 Aug  5  2020 src
```

List Folders

/usr/; cat /etc/passwd | Submit

```
total 88
drwxr-xr-x   2 root root 36864 Sep  6 08:27 bin
drwxr-xr-x   2 root root  4096 Apr 15  2020 games
drwxr-xr-x  36 root root  4096 Aug  4 15:27 include
drwxr-xr-x  77 root root  4096 Aug  1 23:15 lib
drwxr-xr-x   2 root root  4096 Aug  5  2020 lib32
drwxr-xr-x   2 root root  4096 Mar 20 15:24 lib64
drwxr-xr-x   4 root root  4096 Feb  3  2021 libexec
drwxr-xr-x   2 root root  4096 Aug  5  2020 libx32
drwxr-xr-x  12 root root  4096 Feb 16  2021 local
drwxr-xr-x   2 root root 12288 Aug  1 23:15 sbin
drwxr-xr-x 119 root root  4096 Aug  4 15:27 share
drwxr-xr-x   2 root root  4096 Aug  5  2020 src
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbir
```

https://www.stackhawk.com/blog/nodejs-command-injection-examples-and-prevention/

Validate Input Syntax

- To ensure input data meets assumptions, e.g., be printable, HTML, email, userid, etc.
- Compare to what is known acceptable or dangerous (white/blacklisting)
- Using regular expressions
- Patterns of characters describe as “allowable input”
- Details vary among languages
- Bad input could be either rejected or altered.
- Input fuzzing
  - The powerful testing method using a large range of randomly generated inputs to test whether program/function correctly handles abnormal, invalid, unexpected inputs.
  - https://owasp.org/www-community/Fuzzing

Interacting with OS

- Programs are executed on the systems under the control of an OS, which the OS:
  - Shares access to the resources
  - Constructs execution environment
  - Includes environment variable and arguments
- The systems have a concept of multiple users with access permissions on resource.
- The programs may access the shared resources (e.g., the same file).

Preventing Race Condition

- Programs may need to access the common system resources.
- Need suitable synchronization mechanisms
- The most common technique is to lock on shared file during access

```
#!/usr/bin/perl
#
$EXCL_LOCK = 2;
$UNLOCK    = 8;
$FILENAME  = "forminfo.dat";

# open data file and acquire exclusive access lock
open (FILE, ">> $FILENAME") || die "Failed to open $FILENAME \n";
flock FILE, $EXCL_LOCK;
… use exclusive access to the forminfo file to save details
# unlock and close file
flock FILE, $UNLOCK;
close(FILE);
```

Figure 11.8 Perl File Locking Example

Safe Temporary Files

- Many programs use temporary files as their dynamic resources.
- The files are often created in common or shared system area.
- Therefore, the files must be unique and not accessed by other programs.
- Question: Which choice is the best solution?
- Create the name using process ID
- Use random names
- More important, once a program has finished using temporary file, it must be closed and unlinked.

```
char *filename;
int fd;
do {
  filename = tempnam (NULL, "foo");
  fd = open (filename, O_CREAT | O_EXCL | O_TRUNC | O_RDWR, 0600);
  free (filename);
} while (fd == -1);
```

Figure 11.9 C Temporary File Creation Example

Handling Program Output

- Input validation is not enough.
- The programs must identify what is permissible output content and filter any possibly untrusted data to ensure that only valid output is displayed.

```php
<?php

$name = 'Divya';

?>

<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <title>Website</title>
    </head>
    <body>
        <p>Hello <?php echo $name; ?>!</p>
    </body>
</html>
```

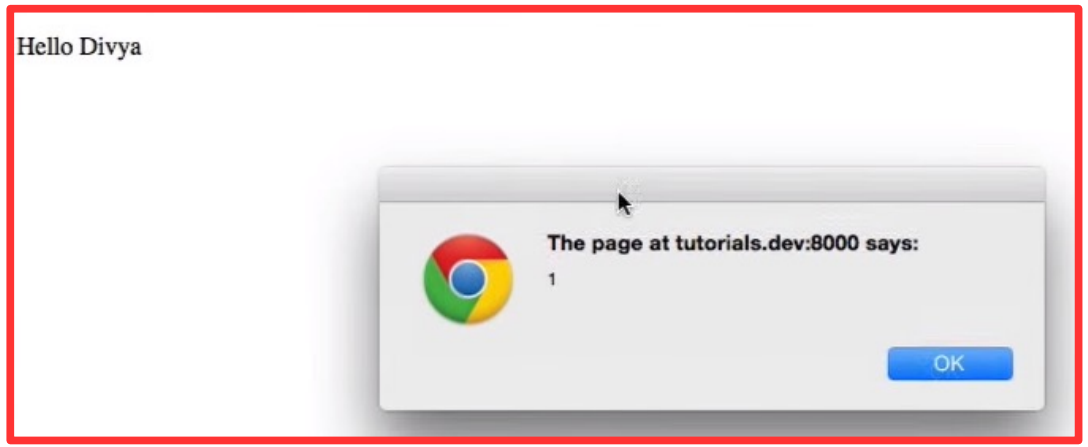
Cross-Site Scripting (XSS) Attack

```php
<?php

$name = 'Divya<script>alert(1);</script>';

?>

<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <title>Website</title>
    </head>
    <body>
        <p>Hello <?php echo $name; ?>!</p>
    </body>
</html>
```



Buffer Overflow

- It's a very common attack mechanism to change the operation flow of program.
- Until now, it is still a major concern as a critical security risk of software application.

1988	The Morris Internet Worm uses a buffer overflow exploit in "fingerd" as one of its attack mechanisms.
1995	A buffer overflow in NCSA httpd 1.3 was discovered and published on the Bugtraq mailing list by Thomas Lopatic.
1996	Aleph One published "Smashing the Stack for Fun and Profit" in *Phrack* magazine, giving a step by step introduction to exploiting stack-based buffer overflow vulnerabilities.
2001	The Code Red worm exploits a buffer overflow in Microsoft IIS 5.0.
2003	The Slammer worm exploits a buffer overflow in Microsoft SQL Server 2000.
2004	The Sasser worm exploits a buffer overflow in Microsoft Windows 2000/XP Local Security Authority Subsystem Service (LSASS).

Buffer Overflow Vulnerabilities

CVE-2026-26269 CNA: GitHub (maintainer security advisories)

Vim is an open source, command line text editor. Prior to 9.1.2148, a stack buffer overflow vulnerability exists in Vim's NetBeans integration when processing the specialKeys command, affecting Vim builds...

Show more

CVE-2026-26011 CNA: GitHub (maintainer security advisories)

navigation2 is a ROS 2 Navigation Framework and System. In 1.3.11 and earlier, a critical heap out-of-bounds write vulnerability exists in Nav2 AMCL's particle filter clustering logic. By publishing a...

Show more

CVE-2026-25994 CNA: GitHub (maintainer security advisories)

PJSIP is a free and open source multimedia communication library written in C. In 2.16 and earlier, a buffer overflow vulnerability exists in PJNATH ICE Session when processing credentials with...

Show more

CVE-2026-25749 CNA: GitHub (maintainer security advisories)

Vim is an open source, command line text editor. Prior to version 9.1.2132, a heap buffer overflow vulnerability exists in Vim's tag file resolution logic when processing the 'helpfile' option....

Show more

CVE-2026-25727 CNA: GitHub (maintainer security advisories)

time provides date and time handling in Rust. From 0.3.6 to before 0.3.47, when user-provided input is provided to any type that parses with the RFC 2822 format, a denial...

Show more

CVE-2026-25646 CNA: GitHub (maintainer security advisories)

LIBPNG is a reference library for use in applications that read, create, and manipulate PNG (Portable Network Graphics) raster image files. Prior to 1.6.55, an out-of-bounds read vulnerability exists in...

Show more

CVE-2026-25584 CNA: GitHub (maintainer security advisories)

iccDEV provides a set of libraries and tools that allow for the interaction, manipulation, and application of ICC color management profiles. Prior to version 2.3.1.3, there is a stack-buffer-overflow vulnerability...

Show more

CVE-2026-25583 CNA: GitHub (maintainer security advisories)

iccDEV provides a set of libraries and tools that allow for the interaction, manipulation, and application of ICC color management profiles. Prior to version 2.3.1.3, there is a heap buffer...

Show more

CVE-2026-25582 CNA: GitHub (maintainer security advisories)

iccDEV provides a set of libraries and tools that allow for the interaction, manipulation, and application of ICC color management profiles. Prior to version 2.3.1.3, there is a heap buffer...

Show more

CVE-2026-25541 CNA: GitHub (maintainer security advisories)

Bytes is a utility library for working with bytes. From version 1.2.1 to before 1.11.1, Bytes is vulnerable to integer overflow in BytesMut::reserve. In the unique reclaim path of BytesMut::reserve,...

Show more

CVE-2026-25506 CNA: GitHub (maintainer security advisories)

MUNGE is an authentication service for creating and validating user credentials. From 0.5 to 0.5.17, local attacker can exploit a buffer overflow vulnerability in munged (the MUNGE authentication daemon) to...

Show more

CVE-2026-25502 CNA: GitHub (maintainer security advisories)

iccDEV provides a set of libraries and tools that allow for the interaction, manipulation, and application of ICC color management profiles. Prior to version 2.3.1.2, stack-based buffer overflow in icFixXml()...

Show more

https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=buffer+overflow

Buffer Overflow

- Programming error when a process attempts to store data beyond the limits of a fixed-sized buffer.
- Overwrites adjacent memory locations
- Locations could hold other program variables, parameters, or program control flow data
- Buffer could be located on the stack, in the heap, or in the data section of the process.

- Consequences
  - Corruption of program data
  - Unexpected transfer of control
  - Memory access violations
  - Execution of code chosen by attacker

Buffer Overflow

```
int main(int argc, char *argv[]) {
    int valid = FALSE;
    char str1[8];
    char str2[8];

    next_tag(str1);
    gets(str2);
    if (strncmp(str1, str2, 8) == 0)
        valid = TRUE;
    printf("buffer1: str1(%s), str2(%s), valid(%d)\n", str1, str2, valid);
}
```

(a) Basic buffer overflow C code

```
$ cc -g -o buffer1 buffer1.c
$ ./buffer1
START
buffer1: str1(START), str2(START), valid(1)
$ ./buffer1
EVILINPUTVALUE
buffer1: str1(TVALUE), str2(EVILINPUTVALUE), valid(0)
$ ./buffer1
BADINPUTBADINPUT
buffer1: str1(BADINPUT), str2(BADINPUTBADINPUT), valid(1)
```

(b) Basic buffer overflow example runs

Figure 10.1 Basic Buffer Overflow Example

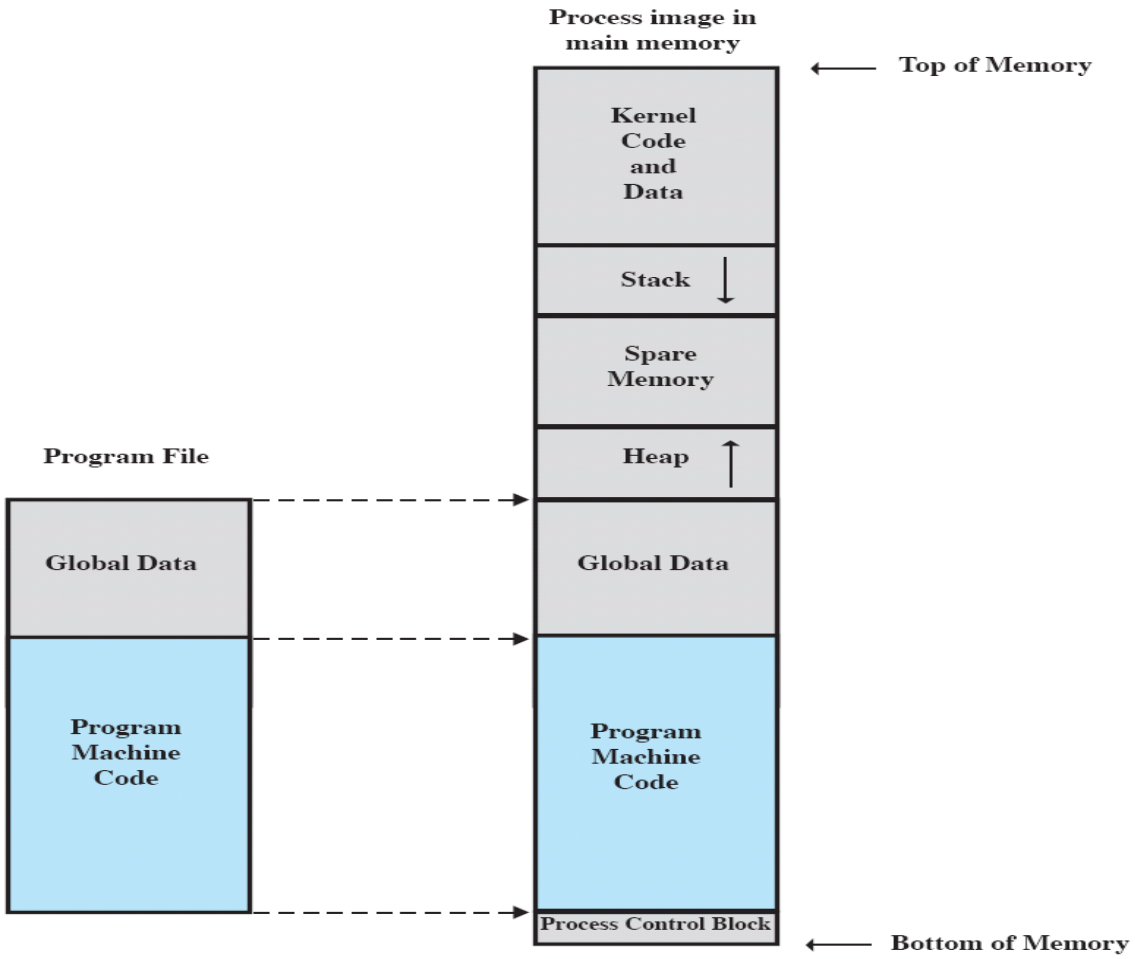
Memory Address	Before gets(str2)	After gets(str2)	Contains Value of
. . . .	. . . .	. . . .	
bffffbf4	34fcffbf 4 . . .	34fcffbf 3 . . .	argv
bffffbf0	01000000	01000000	argc
bffffbec	c6bd0340 . . . @	c6bd0340 . . . @	return addr
bffffbe8	08fcffbf	08fcffbf	old base ptr
bffffbe4	00000000	01000000	valid
bffffbe0	80640140 . d . @	00640140 . d . @	
bffffbdc	54001540 T . . @	4e505554 N P U T	str1[4-7]
bffffbd8	53544152 S T A R	42414449 B A D I	str1[0-3]
bffffbd4	00850408	4e505554 N P U T	str2[4-7]
bffffbd0	30561540 0 V . @	42414449 B A D I	str2[0-3]
. . . .	. . . .	. . . .	

Figure 10.2 Basic Buffer Overflow Stack Values

Buffer Overflow Attack

- To exploit a buffer overflow, an attacker needs:
  - To identify a buffer overflow vulnerability in a program
  - To understand how that buffer is stored in memory and determine potential for corruption
- Identifying vulnerable programs can be done in many ways
  - Inspection of program source
  - Tracing the execution of programs as they process oversized input
  - Using tools such as fuzzing to automatically identify potentially vulnerable programs
- Two categories: stack buffer overflow and heap overflow

Stack Buffer Overflow



- Aka. Stack Smashing
- Occur when buffer is located on stack.
- Exploits included an unchecked overflow of data in stack buffer.
- It is still being a widely exploited.

Stack Buffer Overflow

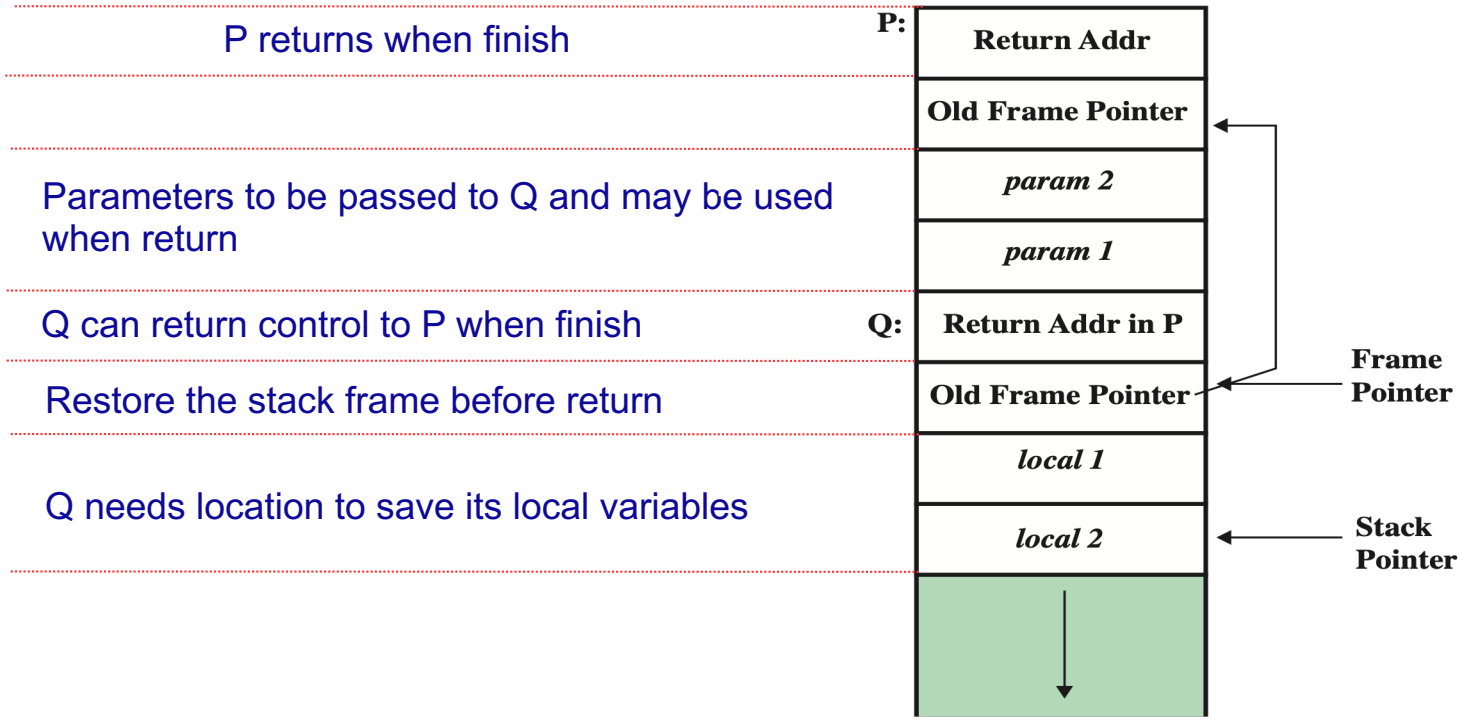


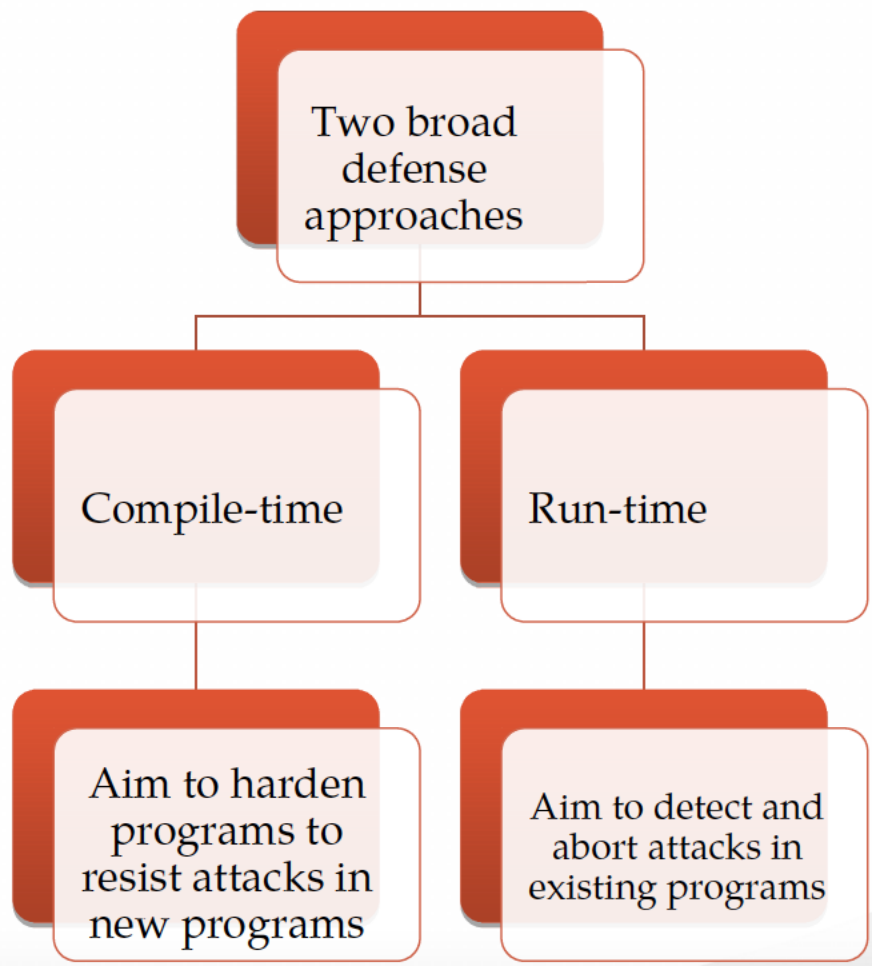
Figure 10.3 Example Stack Frame with Functions P and Q

Unsafe C Standard Libraries

`gets(char *str)`	read line from standard input into str
`sprintf(char *str, char *format, ...)`	create str according to supplied format and variables
`strcat(char *dest, char *src)`	append contents of string src to string dest
`strcpy(char *dest, char *src)`	copy contents of string src to string dest
`vsprintf(char *str, char *fmt, va_list ap)`	create str according to supplied format and variables

When we use these standard function, we have to make sure that size of input variable does not exceed the size of buffer.

Defense Against Stack Overflow



Compile-Time Defenses: Programming Language

- Use a modern high-level language
  - Not vulnerable to buffer overflow attacks
  - Compiler enforces range checks and permissible operations on variables.
- Disadvantages
  - Additional code must be executed at run time to impose checks.
  - Flexibility and safety comes at a cost in resource use.

Compile-Time Defenses: Language Extension/Safe Libraries

- Dynamically allocated memory is hard to handle since the size information is not available at compile time.
- This technique requires:
  - Requires an extension and the use of library routines.
  - Programs and libraries need to be recompiled.
  - Replace unsafe library with safer one.
- Problems: Likely to have problems with third-party applications.

Compile-Time Defenses: Stack Protection

- Add function at entry and exit code to check stack for signs of corruption.
- This feature also known as stack shield and Return Address Defender (RAD)
  - Function entry writes a copy of the return address to a safe region of memory.
  - Function exit code checks the return address in the stack frame against the saved copy.
  - If a change is found, aborts the program.
- Use random canary to check overflow signs.

Run-Time Defenses: Executable Address Space Protection

- Makes some regions of memory non-executable.
- Requires support from memory management unit (MMU).
- Long existed on SPARC/Solaris systems.
- Recent on x86 and x64 Linux/Unix/Windows systems.
- Issues: Special provisions are needed (it is an OS feature, and it is required to turn on before executions).

Run-Time Defenses: Address Space Layout Randomization (ASLR)

- Manipulate location of key data structures
  - Global data ad code section
  - Using random shift for each process and once they are executed
  - Large address range on modern systems means wasting some has negligible impact.
- Randomize location of heap buffers.
- Randomize location of standard library functions.

Run-Time Defenses: Address Space Layout Randomization (ASLR)

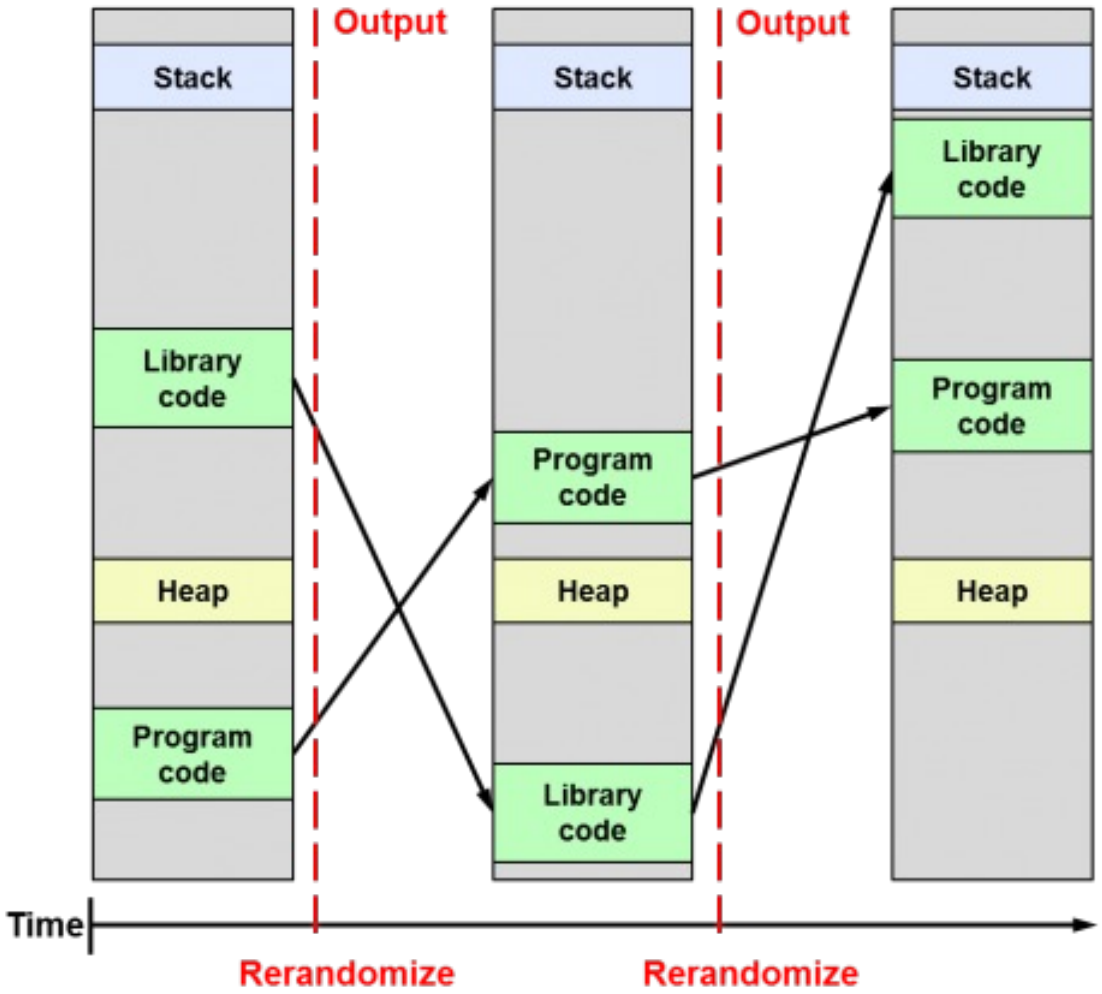
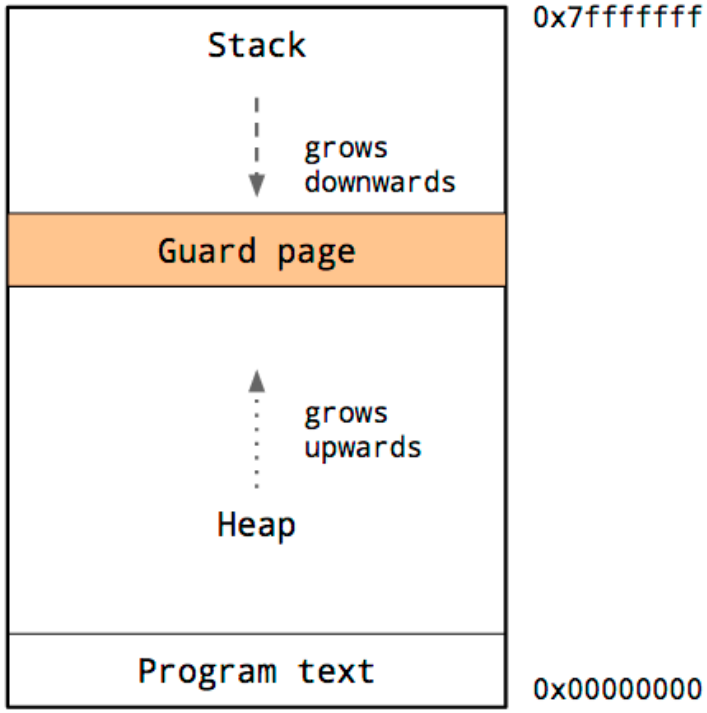


Image: https://www.daniloaz.com/en/differences-between-aslr-kaslr-and-karl/

Run-Time Defenses: Guard Pages

- Place guard pages between critical regions of memory
  - Flagged in MMU as illegal addresses.
  - Any attempted access aborts process.
- Further extension places guard pages between stack frames and heap buffers
  - Cost in execution time to support the large number of page mappings.



Heap Overflow

- The attack buffer is in the heap.
  - Typically located above program code
  - Memory is requested by programs to use in dynamic data structures (such as linked lists of records)
- However, heap does not have return address as stack.
  - Hence, there is no easy transfer of control
  - But may have function pointers that could be exploited.
  - Or manipulate management data structures.

Defense against Heap Overflow

- The protection methods are same as using on stack
- Make the heap non-executable.
- Randomize the allocation of memory on the heap.